

Coding & Solution

Date	10 july 2026
Team ID	
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/sahasraanumolu-cyber/Emotion-Detection-Learning-Support-Engine
Programming Language(s)	Python 3.11, HTML5, CSS3, JavaScript
Framework(s) Used	Flask, TensorFlow/Keras, Hugging Face Transformers, Google Gemini API

Key Features Implemented	• Emotion detection using DistilBERT and BiLSTM models. • Text preprocessing and tokenization. • Detection of 7 emotions (Joy, Sadness, Anger, Fear, Love, Surprise, Neutral). • AI-powered personalized learning recommendations using Gemini API. • Emotion prediction history logging. • User-friendly Flask web interface. • Model comparison and hybrid prediction support
Pending / Incomplete Features	• User authentication module. • Database integration for storing user history. • Deployment monitoring dashboard.
Setup / Run Instructions	1. Clone the GitHub repository. 2. Install dependencies using pip install -r requirements.txt. 3. Configure the Gemini API key in the environment. 4. Run the application using python app.py. 5. Open the generated localhost URL in a web browser.



Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	yes
2	Meaningful variable and function names are used	yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	yes
5	The application runs without critical errors	yes
6	Code is committed to a version control repository	yes

Additional Notes / Comments:

The project follows a **modular software architecture**, separating preprocessing, emotion classification, recommendation generation, prediction logging, and user interface into independent components. Version control is maintained using **GitHub**, and large machine learning models are managed through **Git LFS**. The implementation adheres to Python coding standards, ensuring readability, maintainability, scalability, and ease of future enhancements.